

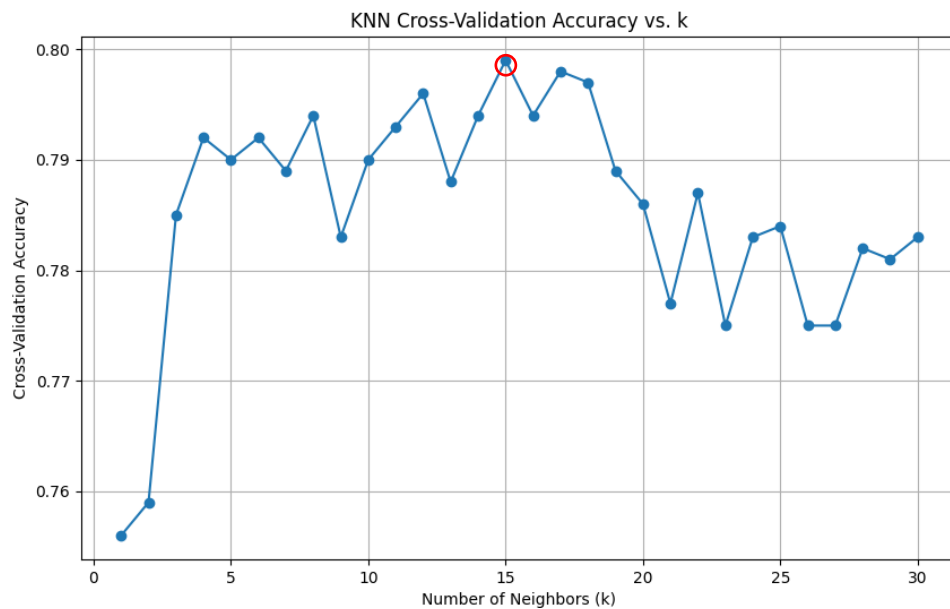
## Assignment 1 – Part 1 | K-Nearest Neighbors

### 1 Methodology

The KNN model was implemented using the `KNNClassifier` class, which calculates Euclidean distances between each test sample and all training sample using the `predict()` method. The class determines predictions based on a majority vote among the  $k$ -nearest neighbors using `np.bincount().argmax()`. The `load_data` function was used to store the contents of the train and test inputs and outputs from comma-delimited files.

To optimize KNN performance, we performed 10-fold cross validation. The dataset was split into 10 subsets using `KFold()` and for each fold, the model is trained on 90% of the data and validated on the remaining 10% and the average accuracy across folds is recorded for each value of  $k$ , ranging from 1 to 30.

### 2 Results and Visualizations



**Figure 1.** Shows a plot of average accuracy based on 10-fold cross-validation when varying the number of neighbors ( $k$ ) from 1 to 30. This visualization helps in understanding how accuracy changes with different  $k$ -values and confirms that  $k=15$  provides the best performance.

Referring to Figure 1, we observe that the best number of neighbours and the corresponding best accuracy occurs in the middle of the  $k$ -range at  $k=15$ , with a corresponding accuracy of **0.799**. However, as  $k$  increases beyond this point, we find that accuracy starts to decline due to excessive smoothing.

After determining the optimal  $k=15$ , we trained the final model on the entire training dataset and evaluated it on the test dataset. The final test accuracy was **0.7727**, which is consistent with our cross-validation results.

### 3 Conclusion

To close, we outlined the implementation of a KNN classifier, including data preprocessing, model training, cross-validation, and final evaluation. These results confirm that  $k=15$  provides optimal accuracy, achieving a test accuracy of **0.7727**.